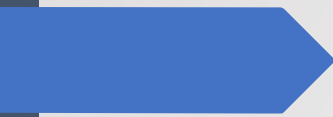




BSM422 Bilgisayarla Görme ve Görüntüleme Teknikleri

Dr. Öğr. Üyesi Mehmet Zahid YILDIRIM



Bu ders;

- Görüntü ön işleme
- Geometrik dönüşümler
- Sayısal Filtreler
- Konvolüsyon
- Eşikleme
- Kenar Tespiti
- Morfolojik İşlemler

Geometrik Dönüşümler

Computer Vision alanında görüntüler çoğu zaman **farklı açılardan, farklı ölçeklerde veya farklı konumlardan** elde edilir. Gerçek dünyada bir nesne:

- kameraya daha yakın veya uzak olabilir (**ölçek değişimi**),
- farklı bir açıdan görülebilir (**dönme**),
- görüntü içinde farklı bir konumda bulunabilir (**öteleme**),

Bu nedenle geometrik dönüşümler:

- görüntü hizalama (**image registration**)
- veri artırma (**data augmentation**)
- nesne takibi (**object tracking**)
- görüntü düzeltme (**image rectification**)

gibi birçok temel işlemde kullanılır.

Matematiksel olarak bu dönüşümler genellikle **matris çarpımı ile ifade edilir**. Bir piksel koordinatı şu şekilde temsil edilir:

$$p = \begin{bmatrix} x \\ y \end{bmatrix}$$

Geometrik Dönüşümler

Öteleme (Translation)

- Bir görüntüyü **x ve y eksenlerinde kaydırma işlemidir.**
- Görüntünün boyutu ve şekli değişmez.

Denklem

$$x' = x + t_x$$

$$y' = y + t_y$$

Homojen koordinat kullanılarak:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Geometrik Dönüşümler

Öteleme (Translation)

```
using System.Drawing;

public static Bitmap TranslateImage(Bitmap inputImage, int tx, int ty)
{
    int width = inputImage.Width;
    int height = inputImage.Height;

    Bitmap outputImage = new Bitmap(width, height);

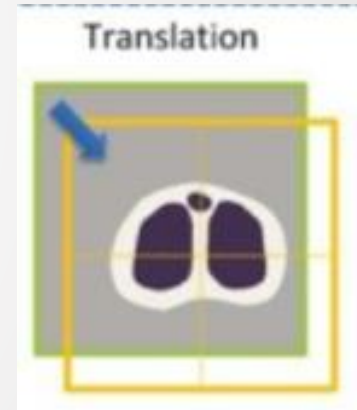
    for (int y = 0; y < height; y++)
    {
        for (int x = 0; x < width; x++)
        {
            int newX = x + tx;
            int newY = y + ty;

            if (newX >= 0 && newX < width && newY >= 0 && newY < height)
            {
                Color pixelColor = inputImage.GetPixel(x, y);
                outputImage.SetPixel(newX, newY, pixelColor);
            }
        }
    }

    return outputImage;
}
```

$$x' = x + t_x$$

$$y' = y + t_y$$



Geometrik Dönüşümler

Öteleme (Translation)

```
public static Bitmap TranslateImageHomogeneous(Bitmap inputImage, int tx, int ty)
{
    int width = inputImage.Width;
    int height = inputImage.Height;

    Bitmap outputImage = new Bitmap(width, height);

    // Homojen dönüşüm matrisi
    int[,] T = {
        {1, 0, tx},
        {0, 1, ty},
        {0, 0, 1}
    };

    for (int y = 0; y < height; y++)
    {
        for (int x = 0; x < width; x++)
        {
            // homojen koordinat vektörü
            int[] p = { x, y, 1 };

            int newX = T[0,0] * p[0] + T[0,1] * p[1] + T[0,2] * p[2];
            int newY = T[1,0] * p[0] + T[1,1] * p[1] + T[1,2] * p[2];

            if (newX >= 0 && newX < width && newY >= 0 && newY < height)
            {
                outputImage.SetPixel(newX, newY, inputImage.GetPixel(x, y));
            }
        }
    }

    return outputImage;
}
```

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Geometrik Dönüşümler

Ölçekleme (Scaling)

- ➔ Görüntüyü **büyütme veya küçültme** işlemidir.

Denklem

$$x' = s_x x$$

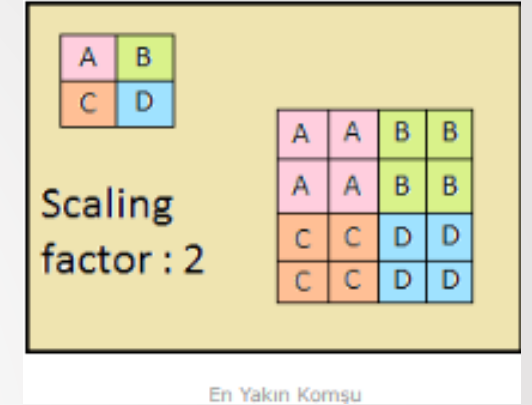
$$y' = s_y y$$

Matris formu:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Homojen form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



- s_x : x yönündeki ölçek
- s_y : y yönündeki ölçek

Durumlar:

- $s > 1 \rightarrow$ büyütme
- $0 < s < 1 \rightarrow$ küçültme

Örnek:

- $s_x = s_y = 2 \rightarrow$ görüntü 2 kat büyür

Geometrik Dönüşümler

Ölçekleme (Scaling)

```
public static Bitmap ScaleImage(Bitmap inputImage, double sx, double sy)
{
    int width = inputImage.Width;
    int height = inputImage.Height;

    int newWidth = (int)(width * sx);
    int newHeight = (int)(height * sy);

    Bitmap outputImage = new Bitmap(newWidth, newHeight);

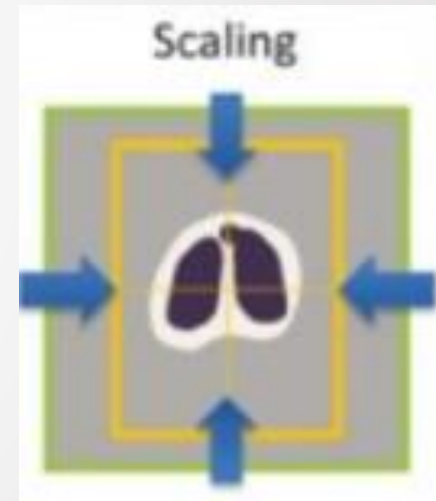
    for (int y = 0; y < newHeight; y++)
    {
        for (int x = 0; x < newWidth; x++)
        {
            int srcX = (int)(x / sx);
            int srcY = (int)(y / sy);

            if (srcX < width && srcY < height)
            {
                outputImage.SetPixel(x, y, inputImage.GetPixel(srcX, srcY));
            }
        }
    }

    return outputImage;
}
```

$$x' = s_x x$$

$$y' = s_y y$$



Geometrik Dönüşümler

Döndürme (Rotation)

- Görüntünün **belirli bir açı kadar döndürülmesidir**.
- Genellikle dönüş görüntü merkezi etrafında yapılır.

Denklem

$$x' = x \cos \theta - y \sin \theta$$

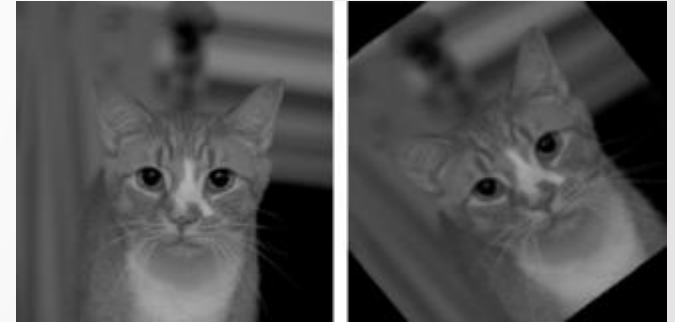
$$y' = x \sin \theta + y \cos \theta$$

Matris formu:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Homojen form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Geometrik Dönüşümler

Döndürme (Rotation)

```
public static Bitmap RotateImage(Bitmap inputImage, double angle)
{
    int width = inputImage.Width;
    int height = inputImage.Height;

    Bitmap outputImage = new Bitmap(width, height);

    double rad = angle * Math.PI / 180.0;

    int cx = width / 2;
    int cy = height / 2;

    for (int y = 0; y < height; y++)
    {
        for (int x = 0; x < width; x++)
        {
            int xt = x - cx;
            int yt = y - cy;

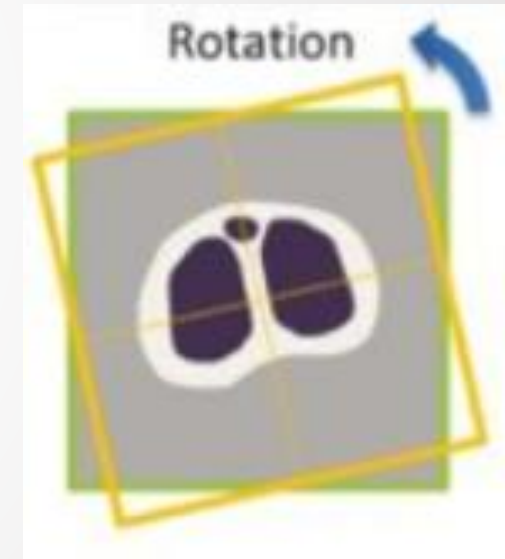
            int xs = (int)(xt * Math.Cos(rad) + yt * Math.Sin(rad)) + cx;
            int ys = (int)(-xt * Math.Sin(rad) + yt * Math.Cos(rad)) + cy;

            if (xs >= 0 && xs < width && ys >= 0 && ys < height)
            {
                outputImage.SetPixel(x, y, inputImage.GetPixel(xs, ys));
            }
        }
    }

    return outputImage;
}
```

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$



Geometrik Dönüşümler

Kesme/Eğme (Shear)

- Görüntünün satırlarını (yatay) veya sütunlarını (dikey) eksenden uzaklıklarına orantılı bir miktarda kaydırarak görüntüyü bozan, eğimli, paralelkenar benzeri bir etki yapan geometrik bir dönüşümdür.



Shear dönüşümü:

- görüntüyü eğerek
- paralel kenar şekline dönüştürür.

Genellikle **kamera açısı değişimlerinin modellenmesinde** kullanılır.

X yönünde shear

$$x' = x + k_x y$$

$$y' = y$$

Matris:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & k_x \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

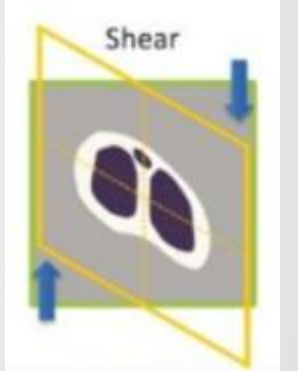
Y yönünde shear

$$x' = x$$

$$y' = y + k_y x$$

Matris:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ k_y & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



Geometrik Dönüşümler

Kesme/Eğme (Shear)

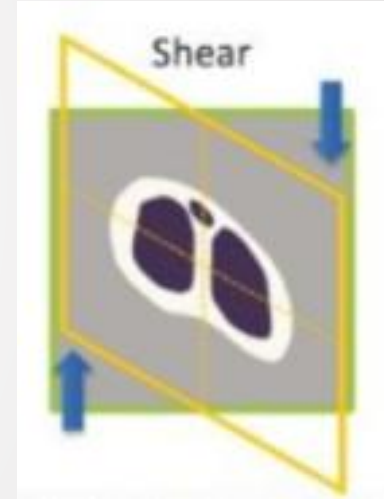
```
public static Bitmap ShearImage(Bitmap inputImage, double shx, double shy)
{
    int width = inputImage.Width;
    int height = inputImage.Height;

    Bitmap outputImage = new Bitmap(width, height);

    for (int y = 0; y < height; y++)
    {
        for (int x = 0; x < width; x++)
        {
            int newX = (int)(x + shx * y);
            int newY = (int)(y + shy * x);

            if (newX >= 0 && newX < width && newY >= 0 && newY < height)
            {
                outputImage.SetPixel(newX, newY, inputImage.GetPixel(x, y));
            }
        }
    }

    return outputImage;
}
```



Sayısal Filtreler

- Sayısal filtreler, görüntü üzerinde belirli bilgileri **güçlendirmek, bastırmak veya ortaya çıkarmak** için kullanılan matematiksel işlemlerdir.
- Filtreler genellikle görüntü üzerinde **komşuluk (neighborhood) temelli işlemler** yapar ve bir pikselin yeni değerini hesaplamak için çevresindeki pikselleri kullanır.
- Bu işlemler genellikle **çekirdek (kernel / mask)** adı verilen küçük matrisler kullanılarak gerçekleştirilir.
- Görüntünün pikselleri üzerinde **bulanıklaştırma, keskinleştirme** veya **gürültü azaltma** gibi dönüşümler yapma işlemidir.

Sayısal Filtreler

Dijital filtreler şu amaçlarla kullanılır:

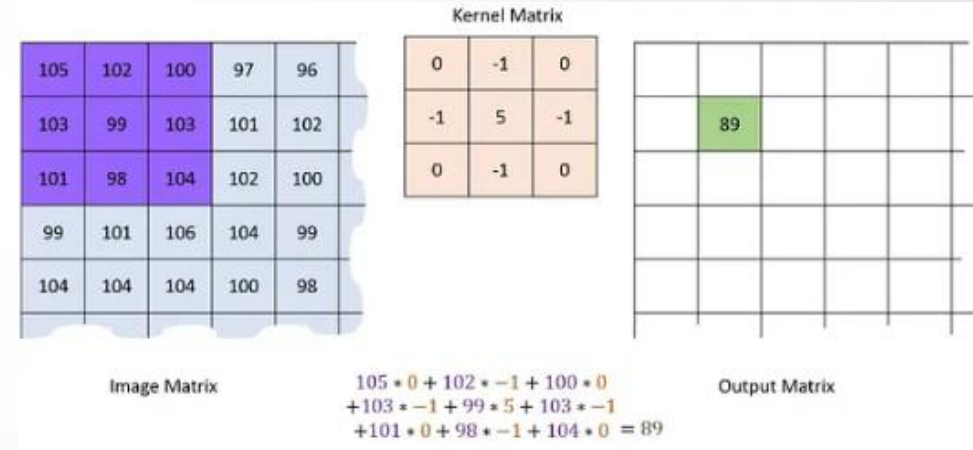
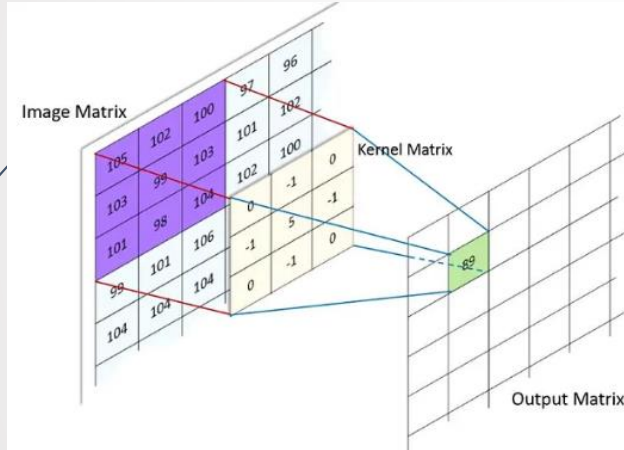
- Gürültü azaltma
- Görüntü yumuşatma
- Kenarları ortaya çıkarma
- Kontrast artırma
- Özellik çıkarımı için hazırlık

Bu nedenle dijital filtreler **kenar bulma, segmentasyon, nesne tanıma ve derin öğrenme tabanlı modellerin** performansı için kritik bir adımdır.

Sayısal Filtreler

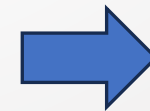
Konvolüsyon İşlemi

- Filtre matrisinin (çekirdek), görüntü üzerindeki piksel değerleriyle çarpılıp toplanarak yeni bir görüntü oluşturulması sürecidir.



Bir görüntü $I(x, y)$ ve kernel $K(i, j)$ için konvolüsyon:

$$G(x, y) = \sum_{i=-k}^k \sum_{j=-k}^k I(x - i, y - j) \cdot K(i, j)$$



Burada:

- $I(x, y) \rightarrow$ giriş görüntüsü
- $K(i, j) \rightarrow$ kernel (filtre matrisi)
- $G(x, y) \rightarrow$ çıktı görüntüsü
- $k \rightarrow$ kernel boyutunun yarısı

Sayısal Filtreler

Konvolüsyon İşlemi

$$G(x, y) = \sum_{i=-k}^k \sum_{j=-k}^k I(x - i, y - j) \cdot K(i, j)$$

```
public static Bitmap Convolution(Bitmap inputImage, double[,] kernel)
{
    int width = inputImage.Width;
    int height = inputImage.Height;

    Bitmap outputImage = new Bitmap(width, height);

    int kernelWidth = kernel.GetLength(0);
    int kernelHeight = kernel.GetLength(1);

    int kCenterX = kernelWidth / 2;
    int kCenterY = kernelHeight / 2;

    for (int y = kCenterY; y < height - kCenterY; y++)
    {
        for (int x = kCenterX; x < width - kCenterX; x++)
        {
            double sum = 0;

            for (int j = -kCenterY; j <= kCenterY; j++)
            {
                for (int i = -kCenterX; i <= kCenterX; i++)
                {
                    Color pixel = inputImage.GetPixel(x + i, y + j);
                    double gray = pixel.R; // gri görüntü varsayımı

                    sum += gray * kernel[i + kCenterX, j + kCenterY];
                }
            }

            int value = (int)Math.Max(0, Math.Min(255, sum));

            Color newColor = Color.FromArgb(value, value, value);
            outputImage.SetPixel(x, y, newColor);
        }
    }

    return outputImage;
}
```


Sayısal Filtreler

Ortalama (Mean)

- Görüntüyü **yumuşatmak (smoothing)** için kullanılır.

Kernel:

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

unfiltered values

5	3	6
2	9	1
8	4	7



mean filtered

*	*	*
*	5	*
*	*	*

$$5 + 3 + 6 + 2 + 9 + 1 + 8 + 4 + 7 = 45$$
$$45 / 9 = 5$$

Orjinal Resim



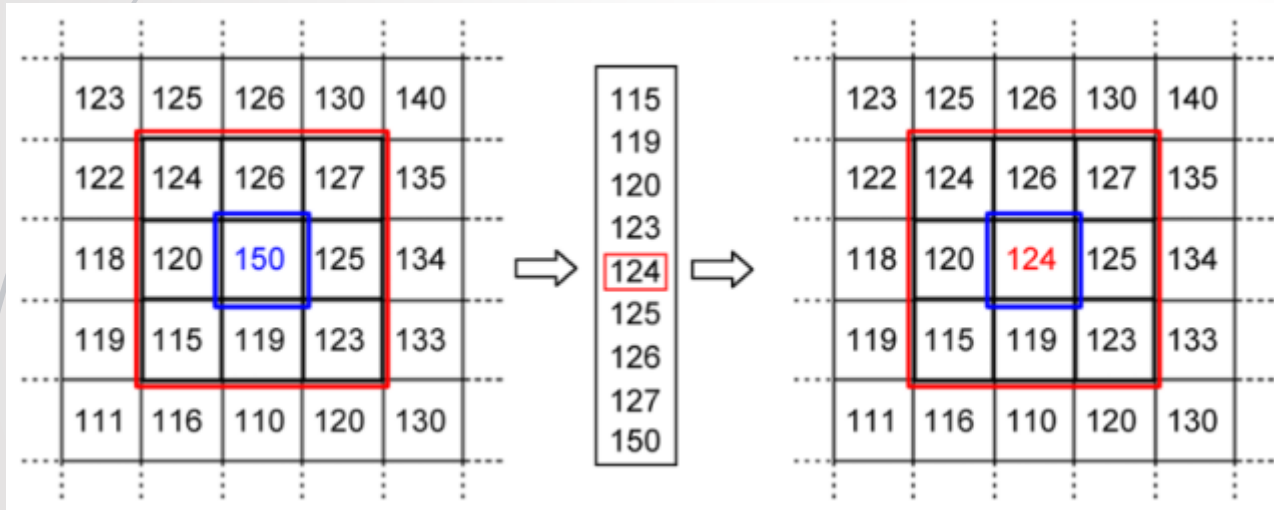
İşlenmiş Resim



Sayısal Filtreler

Ortanca (Median)

- Özellikle **salt & pepper noise** için kullanılır.



Orjinal Resim



İşlenmiş Resim



Sayısal Filtreler

Gauss

- Daha doğal bir **yumuşatma** sağlar.

Kernel:

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Orjinal Resim



İşlenmiş Resim



Sayısal Filtreler

Laplas (Laplacian)

► **Kenarları vurgulamak** için kullanılır.

Kernel:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$



Orjinal Resim

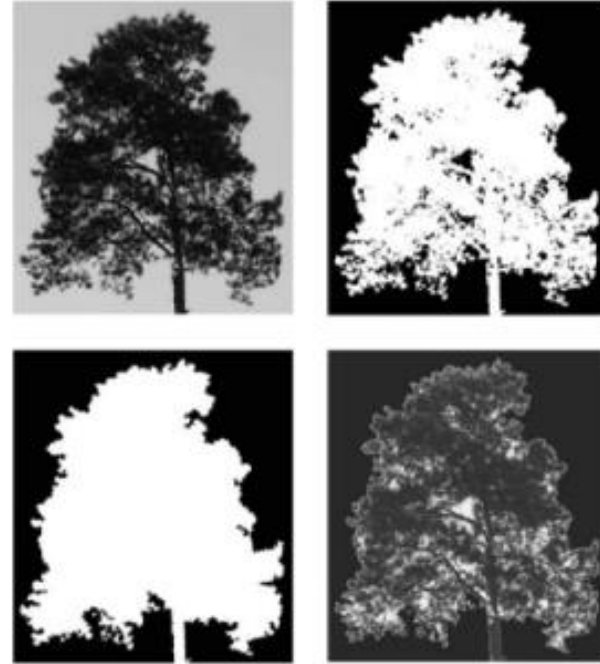
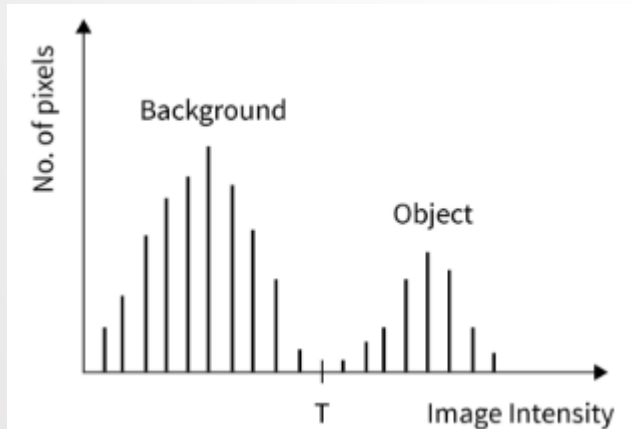


İşlenmiş Resim



Eşikleme (Thresholding)

- Eşikleme, bir görüntüyü anlamlı bölgelerine ayırmak için kullanılan en temel segmentasyon yöntemlerinden biridir.
- Amaç, piksel değerlerini bir eşik değeri ile karşılaştırarak görüntüyü genellikle iki sınıfa ayırmaktır.
 - Nesne (foreground)
 - Arka plan (background)



Eşikleme (Thresholding)

- Gri seviye bir görüntüde her pikselin bir yoğunluk değeri vardır:

$$I(x, y) \in [0, 255]$$

- Thresholding işleminde bir **eşik değeri (T)** belirlenir.

Yeni görüntü şu şekilde oluşturulur:

$$g(x, y) = \begin{cases} 1 & I(x, y) \geq T \\ 0 & I(x, y) < T \end{cases}$$

Çoğunlukla **nesne tespiti ve **segmentasyon** için kullanılır.

Eşikleme (Thresholding)

► Thresholdin görüntülere birkaç farklı şekilde uygulanabilir. Bunlar:

- **Global Thresholding**

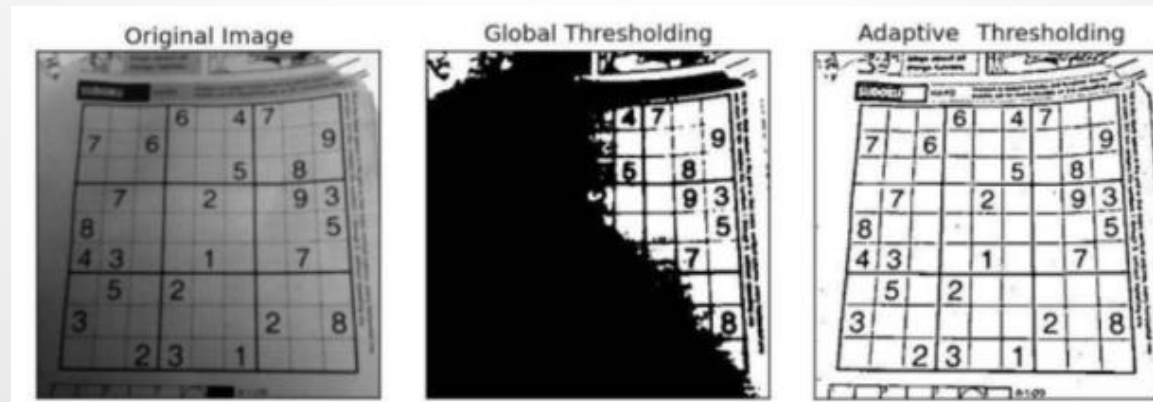
- ✓ Tek bir eşik değeri kullanılır.
- ✓ Tüm Görüntü için aynıdır.

► $T = \text{sabit}$

- **Adaptive Thresholding**

- ❖ Eşik değeri görüntünün her bölgesi için ayrı hesaplanır.
- ❖ Yani her piksel için çevresindeki pikseller kullanılır.
- ❖ Gölge görüntüler ve doğal sahnelerde etkilidir.

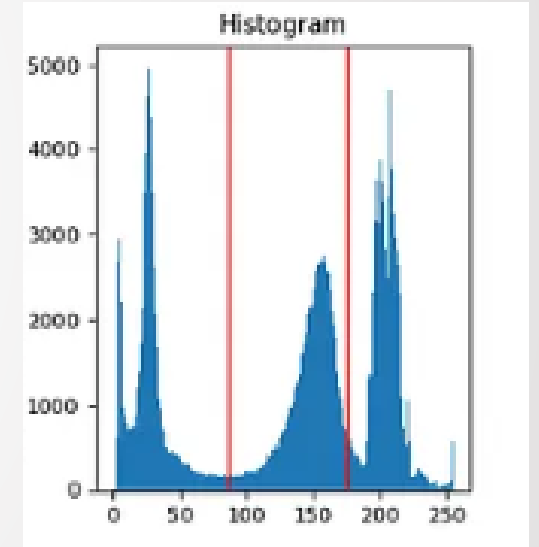
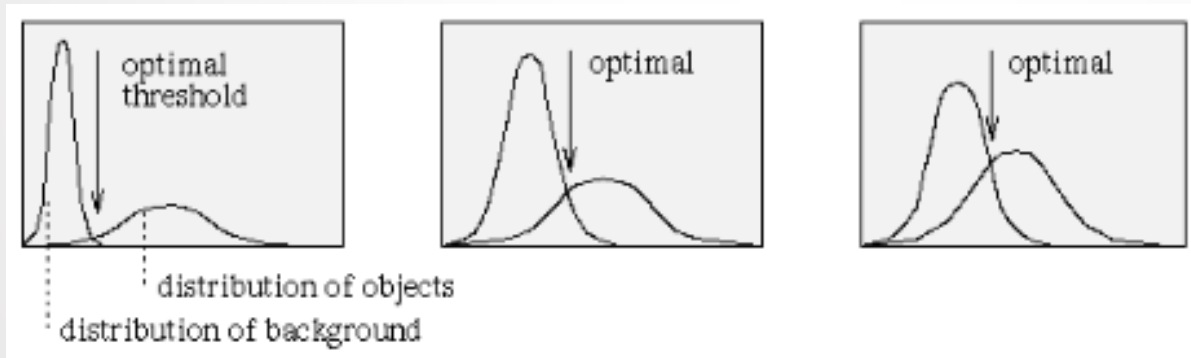
► $T = f(\text{local neighborhood})$



Eşikleme (Thresholding)

- **Otsu Thresholding**

- Eşik değeri otomatik olarak hesaplanır.
- Bu yöntem histogramı analiz ederek en iyi eşik değerini bulur.
- Amaç sınıflar arası varyansı maksimize etmektir.



Histogram **bimodal (iki tepe varsa) ise Otsu yöntemi çok iyidir.

Kenar Tespiti (Edge Detection)

- Kenar tespiti görüntü ön-işlemenin en kritik aşamalarından biridir.
- Çünkü bir görüntüdeki **nesne sınırları**, **şekil bilgisi** ve **yapısal özellikler** çoğunlukla kenarlarda bulunur.
- Segmentasyon, nesne tanıma, takip ve feature extraction işlemleri genellikle kenar bilgisi üzerine kuruludur.
- Bir görüntüde kenar, **Yoğunluk (Intensity)** değerinin ani değiştiği bölgelerdir.

$$Edge \Rightarrow \frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \text{ büyük}$$

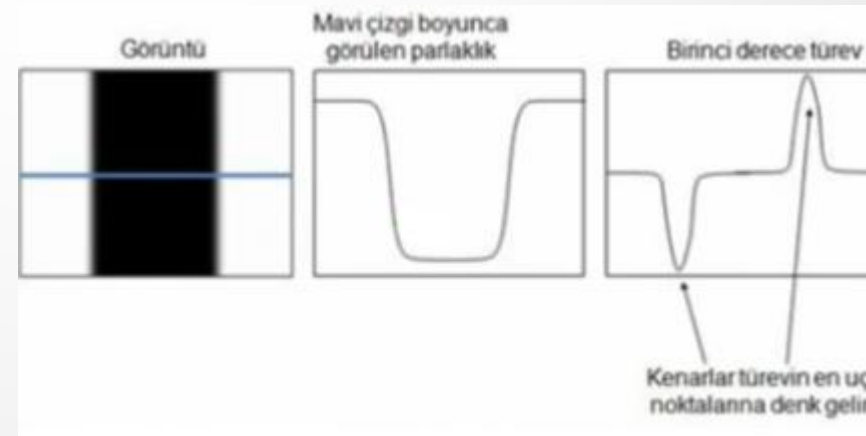
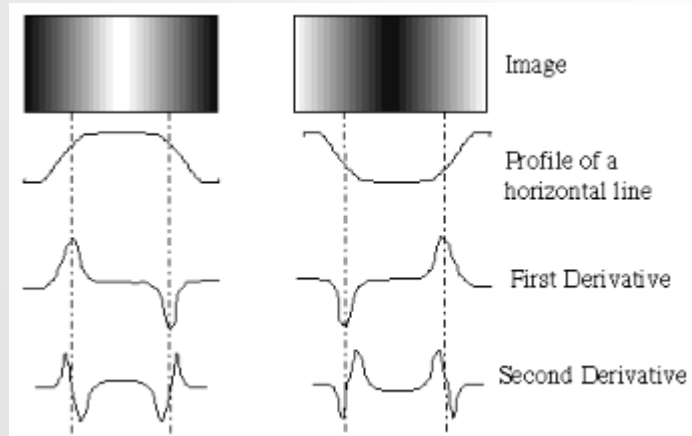
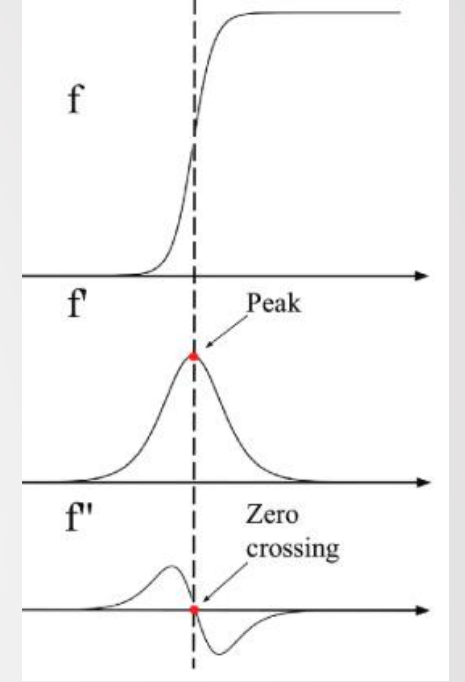
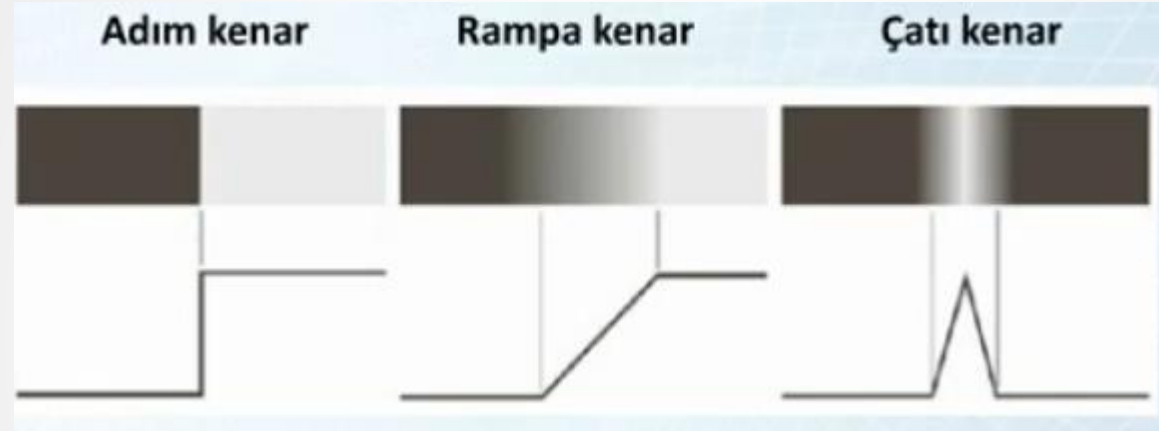
**Yani: Piksel değerinin türevi yüksekse → kenar vardır.

Matematikte türev:

$$f'(x) = \text{değişim hızı}$$

Kenar Tespiti (Edge Detection)

Kenar Türleri ve Türev



Kenar Tespiti (Edge Detection)

Algoritmalar (1. Türev Tabanlı)

X ve Y yönü Çekirdekler

➤ Roberts

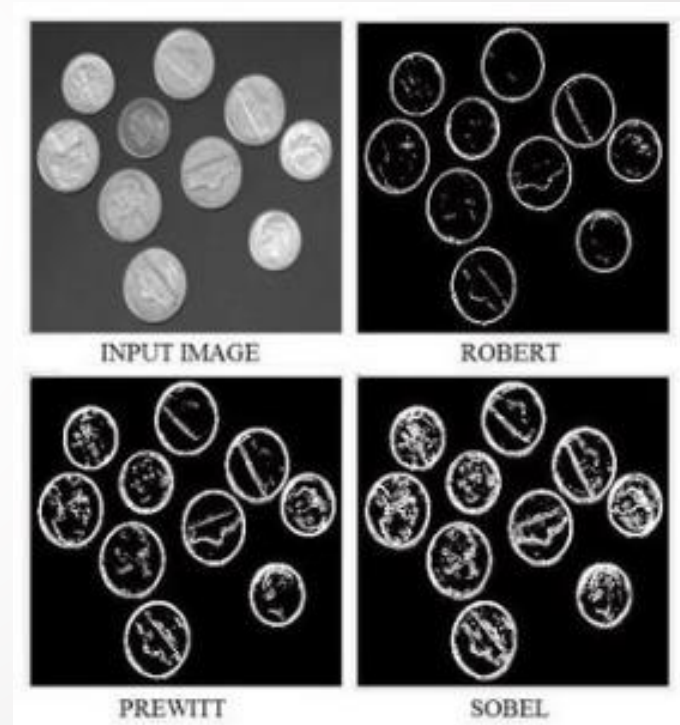
$$G_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

➤ Prewitt

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$



Kenar Tespiti (Edge Detection)

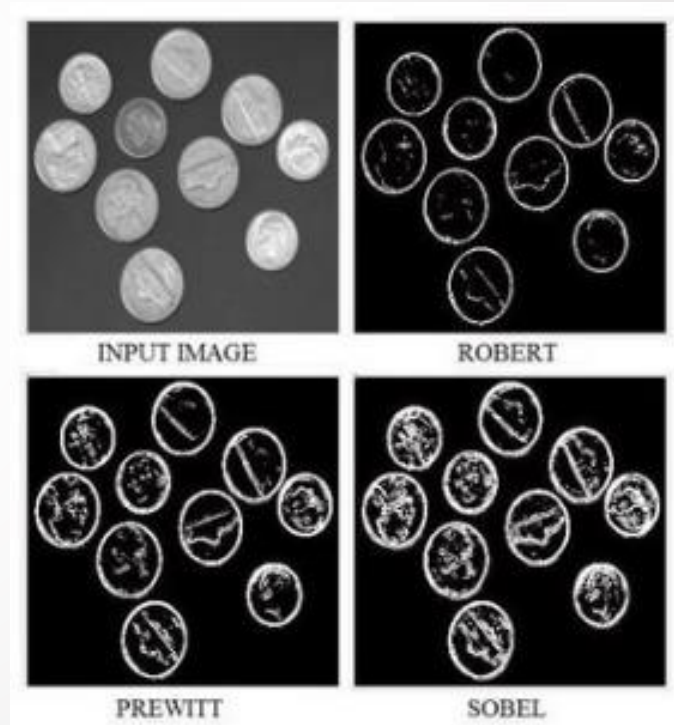
Algoritmalar (1. Türev Tabanlı)

➔ Sobel

X ve Y yönü Çekirdekler

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$



Kenar Tespiti (Edge Detection)

Algoritmalar (2. Türev Tabanlı)

➤ Laplacian

4-komşuluk

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

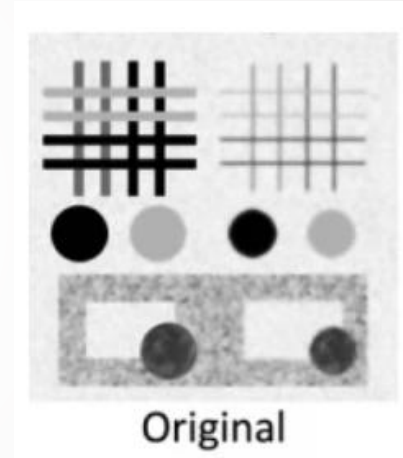
8-komşuluk

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

➤ Laplacian of Gaussian (LoG)

$$\begin{bmatrix} 0 & 0 & -1 & 0 & 0 \\ 0 & -1 & -2 & -1 & 0 \\ -1 & -2 & 16 & -2 & -1 \\ 0 & -1 & -2 & -1 & 0 \\ 0 & 0 & -1 & 0 & 0 \end{bmatrix}$$

Gaussian Blur → Laplacian



Kenar Tespiti (Edge Detection)

Canny Algoritması (Hibrit)

1. Smooth image (Gaussian)

→ Gürültünün türev işlemini bozmasını engellemek için görüntü Gaussian filtresi ile yumuşatılır.

2. Compute gradient (Sobel)

→ Sobel operatörü kullanılarak her pikseldeki yoğunluk değişim miktarı ve kenar yönü hesaplanır.

3. Thin edges (Non-Maximum Suppression - NMS)

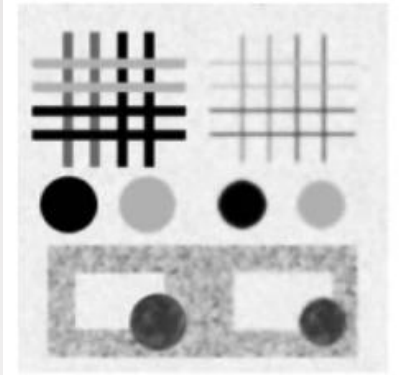
→ Gradient yönünde yalnızca yerel maksimum piksel değerleri korunarak kalın kenarlar tek piksel genişliğine inceltilir.

4. Classify edges (Double Threshold)

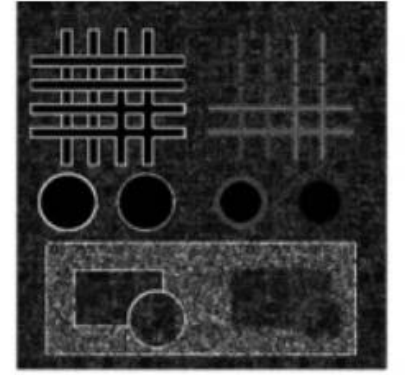
→ Kenar pikselleri güçlü, zayıf ve gürültü olarak sınıflandırılmak üzere iki farklı eşik değeri ile değerlendirilir.

5. Connect edges (Hysteresis)

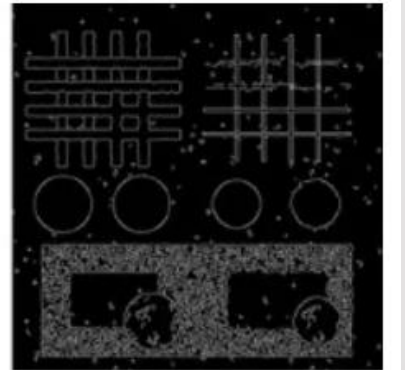
→ Güçlü kenarlara bağlı olan zayıf kenarlar korunarak sürekliliği olan gerçek kenarlar oluşturulur.



Original



Laplacian

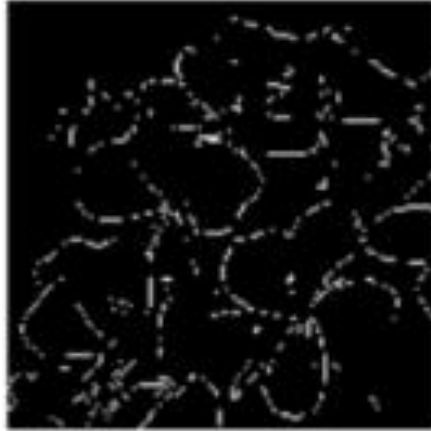


Canny

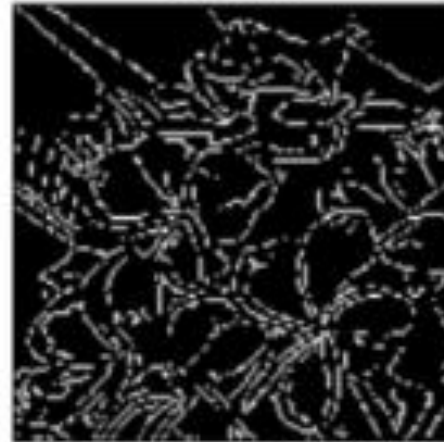
Gray Scale Image



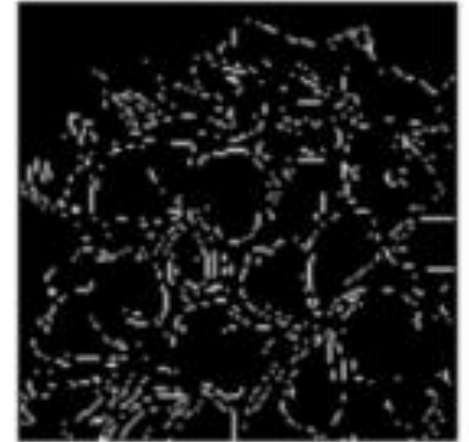
Sobel



Canny



Log



Original Image



Sobel Edge Detection



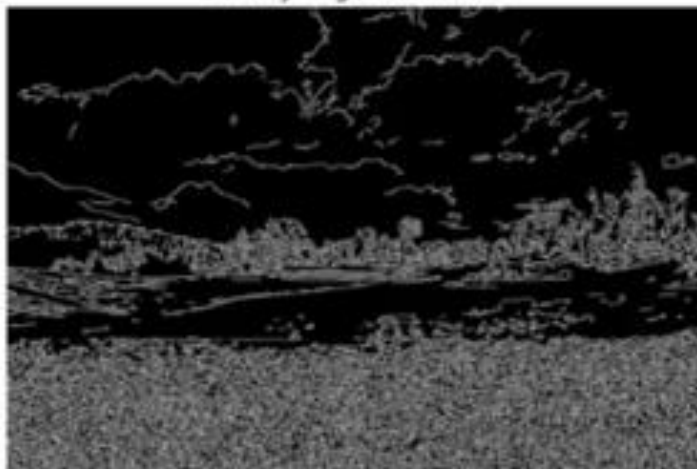
Prewitt Edge Detection



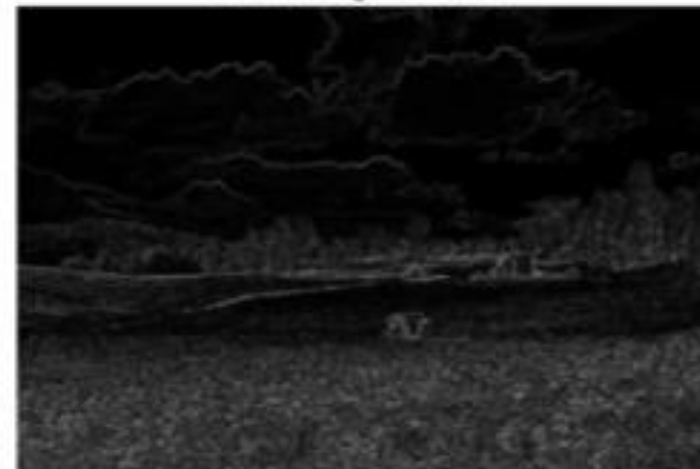
Laplacian of Gaussian



Canny Edge Detection



Roberts Edge Detection



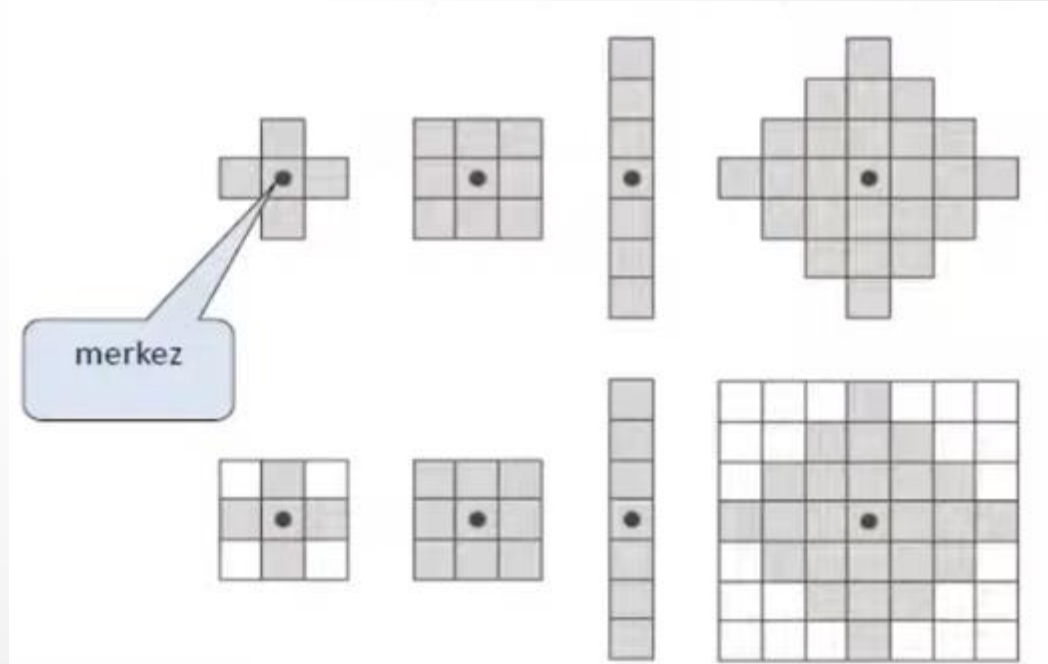
Morfolojik İşlemler

- Morfolojik işlemler, esas olarak ikili görüntülerde gürültüyü temizlemek, sınırları geliştirmek ve yapıları analiz etmek için kullanılan şekil tabanlı görüntü işleme teknikleridir.
- Özellikle şu işlemlerde kullanılır:
 - Gürültü temizleme,
 - Nesnelerin sınırlarını düzeltme,
 - Küçük boşlukları doldurma,
 - Nesneleri ayırma veya birleştirme,
 - Segmentasyon sonrası iyileştirme (post-processing)

Morfolojik İşlemler

Yapısal Eleman

- Piksel değerlerinden çok şekil/yapı (structure) ile ilgilenir.
- Görüntü, küçük bir **yapısal eleman (structuring element)** ile taranır ve komşuluk ilişkisine göre piksel güncellenir.
- Farklı yapısal eleman modelleri:



Morfolojik İşlemler

Aşındırma (Erosion) ve Yayma (Dilation)

- Morfolojik işlemler temelde iki operasyondan oluşur.

Aşındırma



$$A \ominus B$$

Yayma

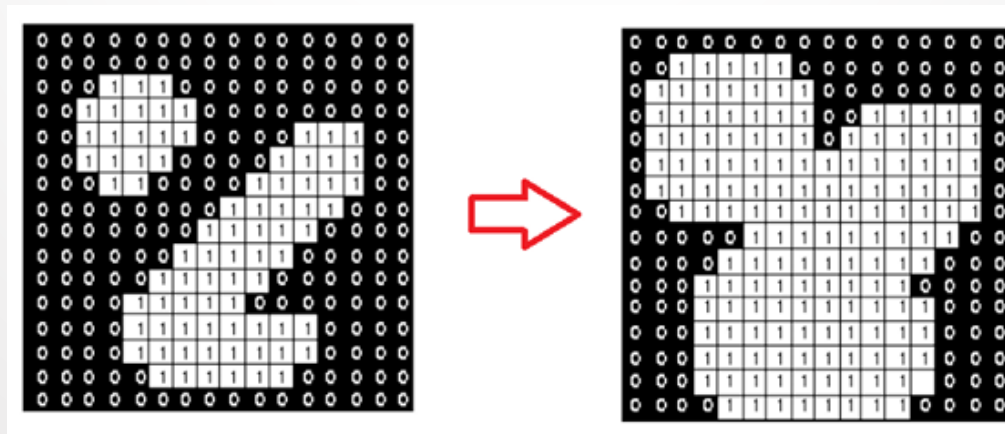
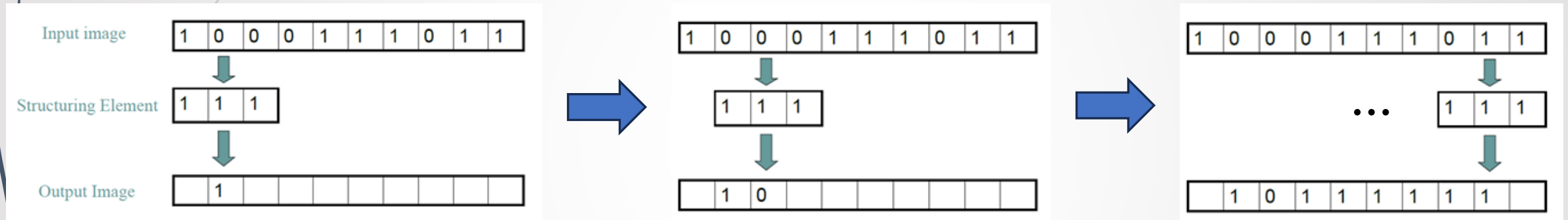


$$A \oplus B$$

Morfolojik İşlemler

Aşındırma (Erosion) ve Yayma (Dilation)

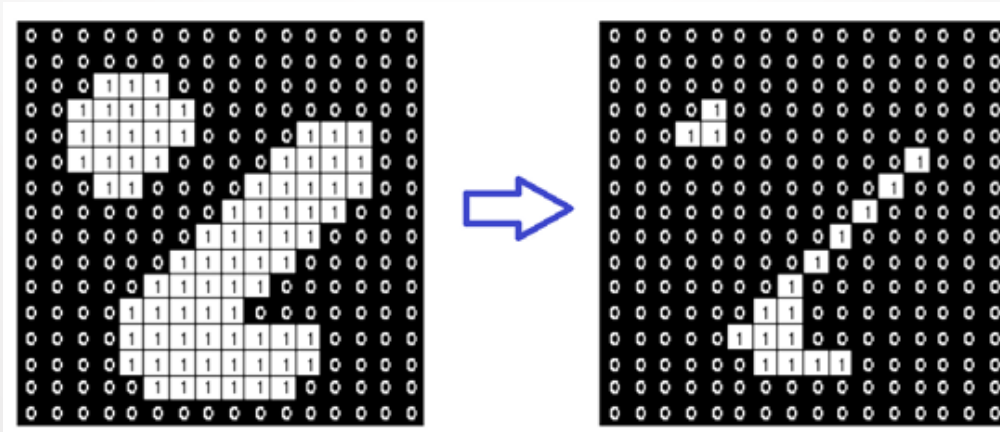
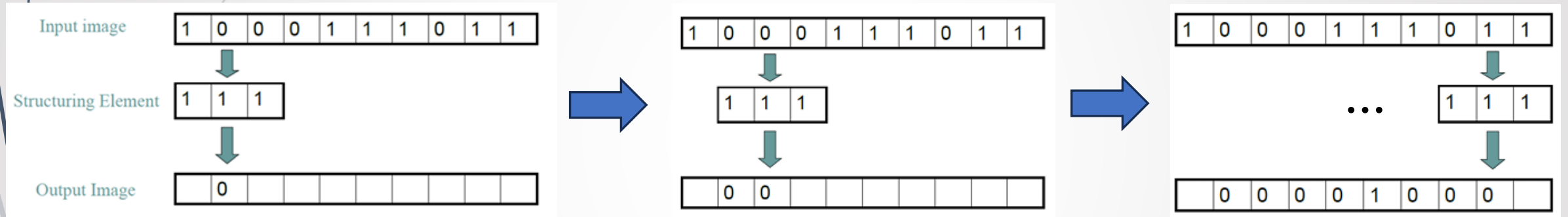
Örnek Yayma (Dilation) İşlemi:



Morfolojik İşlemler

Aşındırma (Erosion) ve Yayma (Dilation)

Örnek Aşındırma (Erosion) işlemi:



Morfolojik İşlemler

Türetilmiş İşlemler

Opening (Açma)	$A \circ B = (A \ominus B) \oplus B$	Küçük gürültüleri kaldırmak ve nesnenin genel şeklini koruyarak ince çıkıntıları yok etmek için kullanılır.
Closing (Kapama)	$A \bullet B = (A \oplus B) \ominus B$	Nesne içindeki küçük boşlukları doldurmak ve kopuk bölgeleri birleştirmek için kullanılır.
Morphological Gradient	$G = (A \oplus B) - (A \ominus B)$	Nesne sınırlarını (kenarlarını) belirlemek için kullanılır.
Top-Hat Transform	$T = A - (A \circ B)$	Arka plandan daha parlak olan küçük nesneleri ortaya çıkarmak için kullanılır.
Black-Hat Transform	$B = (A \bullet B) - A$	Arka plandan daha koyu olan küçük bölgeleri tespit etmek için kullanılır.

Morfolojik İşlemler

Türetilmiş İşlemler

